

COMPILER DESIGN (Yt:- Raghavendra Babu)

19/6/26

UNIT-1:-

- Compiler is a translator that converts source language into target language.

source $\rightarrow$ Compiler $\rightarrow$ Target

Ex:- C language

Basic code:-
(5 lines)

```
#include <stdio.h>
int main()
{
    printf("Hello");
}
```

But, no. of lines compiled is 1018 : because

"printf" is HLL (high level language) function.

Here, relevant codes will be replaced in our code from '.h' included files.

Then after all replacements and preprocessing, that code will be compiled. Hence, 1018 lines.

First step after compilation starts, is preprocessing.

And this is why we need to optimize the code.

Compiler phases:- Block diagram/Architecture

Lexical analyzer I

Syntax analyzer II

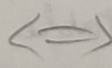
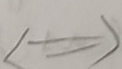
Semantic analyzer III

Intermediate code generator

Code optimizer

Code generator

Symbol Table



Error Table

} IV, V

HLL $\rightarrow$ Preprocessing $\xrightarrow{\text{Pure HLL}}$ Compiler.

- Compiler is divided into 6 phases.
and these phases are divided as 2 passes:-

- Frontend pass :- first 4 phases
- Backend pass :- last 2 phases.

Note:- Code optimization phase is optional.

Even if code is not optimized, it still works.

- * The compiler first eliminates spaces and comments in the input (as str)

Ex: $x = a + b$

Then it splits them as lines (based on variables, operators, symbols, keywords etc). These are called as **TOKENS**.

Ex: $x = a + b$

$$x = a + b \cdot i$$
$$n_1 = n_2 + n_3$$
$$n_1 + n_2 + n_3 + \dots + n_j$$

* ; represents end of a statement

Lexical analyzer :-

- Scans / analyzes the lines as what are variables, operators etc in the program

- It identifies these tokens and stores in the 'symbol table' (which is empty initially)

— These identified tokens are called as "Lexemes".

- Lexical analyzer is also known as "Scanner".

- The line taken from code file will be first loaded into a separate memory called "Input Buffer".

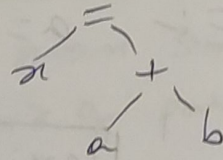
Then the work on that line happens from this memory.

- The lexemes are sent to the syntax analyzer which counts them into parse tree and parse table.

- It uses parser algorithms and context free grammar for parse tree generation

Ex: - $x = a + b;$

removed
after identification



* Leaf nodes of parse tree must be variables

- Grammar is used to identify validity of the statement using syntax (rules) in that language.

- As the input is being parsed, this phase is called 'parser', and the output is parse tree.

* Lexical analyzer uses Regular expressions (RE)

- RE help in identifying tokens and store them

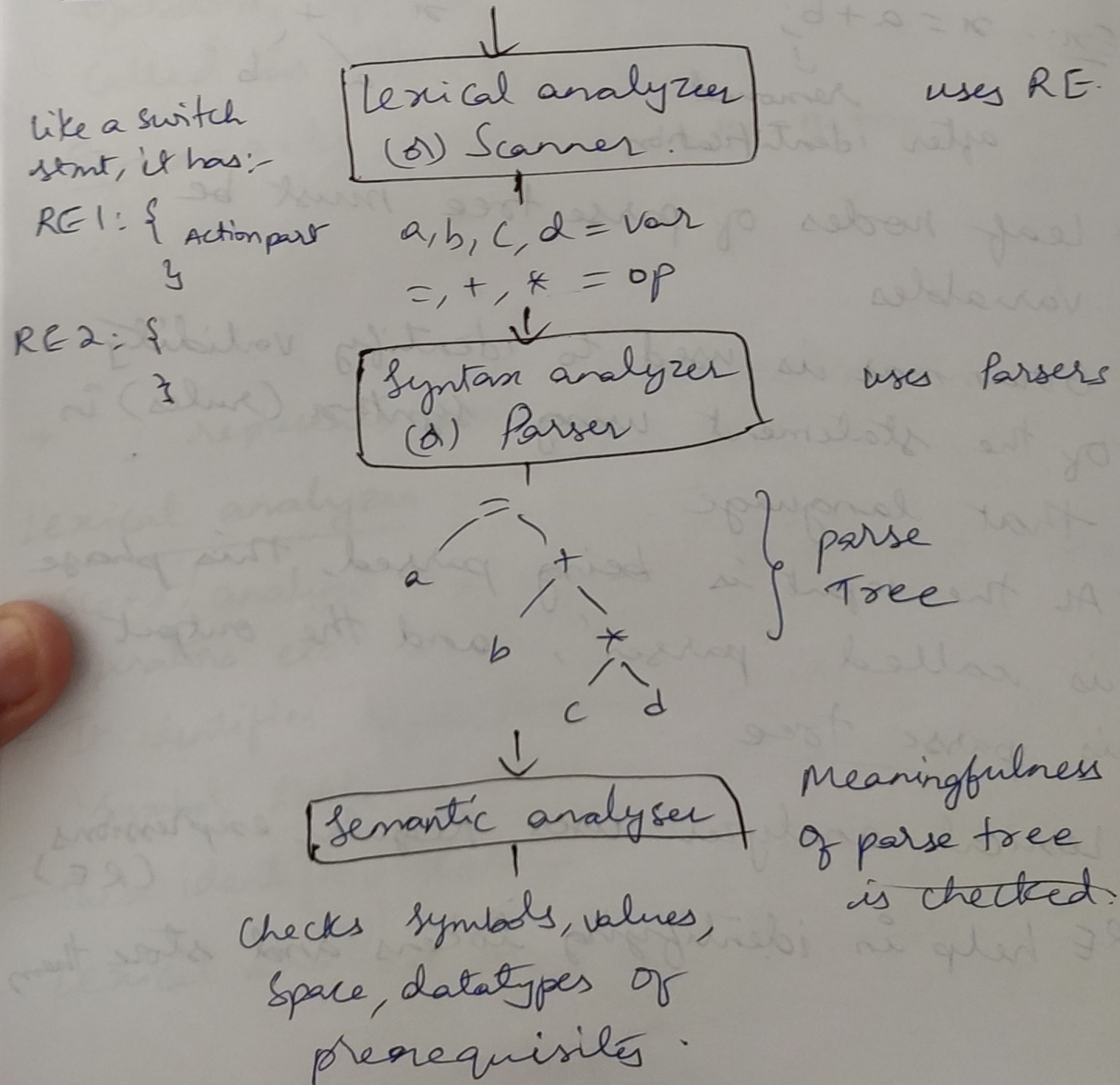
- C has a character array which has all rules written in 'c' for variables like starts with _ and alphabets etc
- Instead RE uses simple code like :-

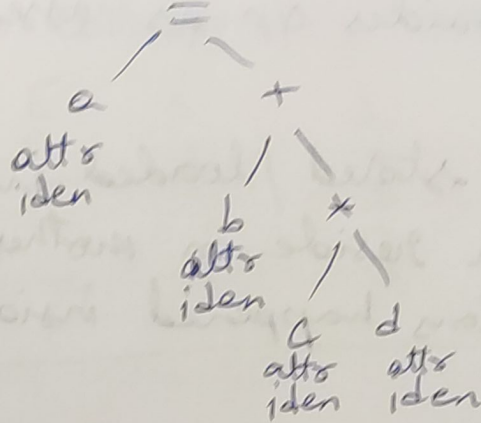
$$[_a-zA-Z][a-zA-Z0-9] \dots$$

↓
1st letter

 to identify a token as variable
- It categorizes variables, constants, keywords, operators etc.

EXAMPLE:- $a = b + c * d$





Absolute
parse tree

(For each identifier, attribute is attached)

Intermediate Code generator

converts parse tree into normal statements

Most commonly used :- 3 address code

(each line can have max 3 variables)

- starts from lowest part of parse tree

$t_1 := c * d$
 $t_2 := b + t_1$
 $a := t_2$

- we add : before = to identify it as intermediate code.

Code optimizer

checks whether we can reduce lines of code.

Ex:- $t_1 := c * d$
 $a := b + t_1$

Code generator (target code generator)

- C uses microprocessors to generate.

MOV C, R0	MOV b, R2
MOV d, R1	ADD R0, R2
MUL R0, R1	MOV R0, a

* C compiler provides Q/P as assembly language.

The values are stored / loaded into registers which reside on motherboard and calculations happened inside.

Lex language:-

— should be stored as .l files.

Structure:-

i) Var creation & declaration (int x=10;)

— preprocessing statements (#include)

ii) Pattern recognition, regular expressions.

— first it has to check if token is keyword / not then variable / not then number / not

printf/scanf. ---

Keywords.

[a-zA-Z][a-zA-Z0-9]*

Variables.

[0-9][0-9]*

Numbers

iii) Optional.

⇒ i) % {
 % }

ii) % %
 % %

Yacc language:-

Same as lex structure but in ii) part, instead of RE, it has grammar statements

and their corresponding action part

$E : E + E$	{	}
$E : E * E$	{	}
$E : id$	{	}

UNIT-2.

Syntax Analysis:-

- Ambiguous grammar are not supported by parses.

- CFG (context free grammar)

{
→ UAG (unambiguous)
→ No left recursion
→ Be left factoring.

} Parses can only work on these.

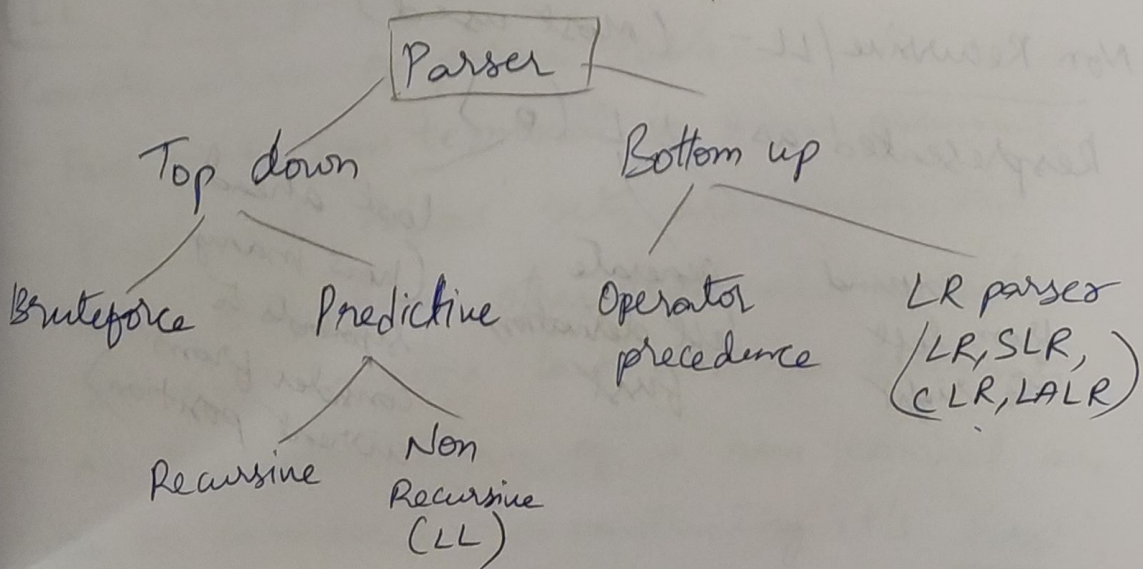
Left Recursion:-

$A \rightarrow A \alpha$
same

⇒ left recursion not supported

(parse tree moves towards left)

$A \rightarrow \alpha A$ ⇒ right recursion are supported.



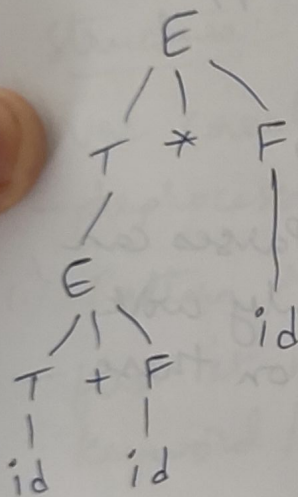
Ex: $E \rightarrow T + F / T * F$
 $T \rightarrow E / id$
 $F \rightarrow id$

string: -

$id + id * id.$

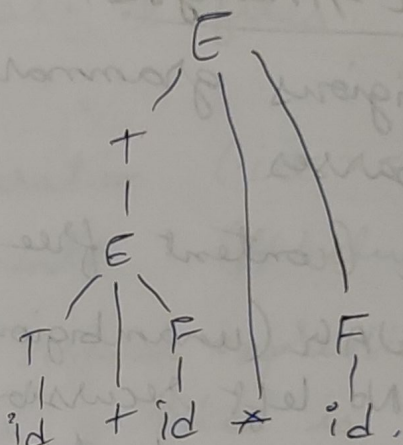
Top down parser

generating I/P
string from start
symbol.



Bottom up parser

deriving start symbol
from input string.



Brutforce - trial & error and backtracking
process.

(not used much because it's not optimal)

Recursive - Some left recursions are supported
so this is also not used much

Non Recursive/LL - (Most used)

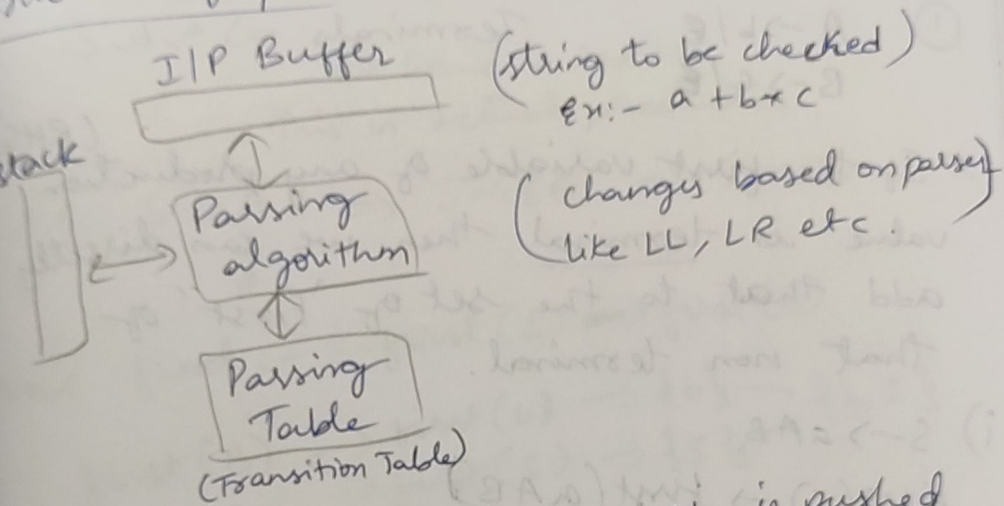
Represented as $LL(0)$

expand
from left
to right

generate
left derivation
first

look ahead
(how many
symbols to
consider from
current position)

Structure of parser :-



The start symbol of grammar is pushed into stack. Ex:- E is pushed

$E \rightarrow E + T$
$T \rightarrow T * F$
$F \rightarrow Id$

The first symbol in I/P buffer is '\$'.

- I/P buffer acts as a queue.

* For each parser, there is different way of constructing parsing table.

* Syntax analyzer uses parser to detect whether the syntax of given string is correct or not based on given grammar.

LL parser :-

Construction of parser table:-

- For each non terminal in given grammar, 'First' and 'Follow' sets are constructed.

These sets have other terminals/symbols

(1) any operators but not non terminals.

- All the productions of a non terminal are to be considered when constructing it's 'First'.

Ex:- $S \rightarrow aAB$

Non terminals $\Rightarrow S, A, B$

① $A \rightarrow b/\epsilon$

Terminals $\Rightarrow a, b, c$

$B \rightarrow c/\epsilon$

(RHS)

* If the first variable of any production value is terminal then we can directly add that to the set of 'First' of that non terminal.

i) $S \rightarrow aAB$

$\text{first}(S) \rightarrow \text{first}(aAB)$

$\downarrow$
Terminal.

$\{a\}$

$A \rightarrow b/\epsilon$

$\text{first}(A) \rightarrow \text{first}(b)$
 $\{b\}$

$\text{first}(A) \rightarrow \text{first}(\epsilon)$
 $\{\epsilon\}$

$\Rightarrow \{b, \epsilon\}$

$B \rightarrow c/\epsilon$

$\text{first}(B) \rightarrow \text{first}(c)$
 $\{c\}$

$\text{first}(B) \rightarrow \text{first}(\epsilon)$
 $\{\epsilon\}$

$\Rightarrow \{c, \epsilon\}$

* If the first variable of any production (RHS) value is non-terminal then we find the first of that non terminal.

Ex:- $S \rightarrow AB$

NT:- S, A, B

② $A \rightarrow a/\epsilon$

T:- a, b

$B \rightarrow b/\epsilon$

i) $S \rightarrow AB$

$\downarrow$
NT

$\text{first}(S) \Rightarrow \text{first}(AB) \Rightarrow \text{first}(A)$

$\text{first}(A) \rightarrow \text{first}(a) \rightarrow \{a\} \rightarrow \{a, \epsilon\}$
 $\text{first}(A) \rightarrow \text{first}(\epsilon) \rightarrow \{\epsilon\} \rightarrow \{a, \epsilon\}$

So, $\text{first}(S) \Rightarrow \{a, \epsilon\}$ now.

$\downarrow$
we have to replace
this is actual production.

$$\text{i.e. } S \rightarrow AB \Rightarrow S \rightarrow \underset{\downarrow \epsilon}{\epsilon} B \Rightarrow S \rightarrow B.$$

Now find: - $\text{first}(S) \Rightarrow \text{first}(B)$

$$\text{first}(B) \begin{cases} \rightarrow \text{first}(b) \rightarrow \{b\} \\ \rightarrow \text{first}(\epsilon) \rightarrow \{\epsilon\} \end{cases} \Rightarrow \{b, \epsilon\}.$$

$$\text{Again replace: - } S \rightarrow B \Rightarrow S \rightarrow \underset{\downarrow \epsilon}{\epsilon}.$$

$$\text{Again find: - } \text{first}(S) \rightarrow \text{first}(\epsilon) \Rightarrow \{\epsilon\}.$$

Final first of S: -

$$\text{first}(S) \Rightarrow \{a, b, \epsilon\} //$$

Rules: - For constructing "First": -

1) If 1st variable of RHS is :-

- i) Terminal $\Rightarrow$ directly add it to set
- ii) Non terminal $\Rightarrow$ find first of that
- iii) ϵ $\Rightarrow$ add ϵ , and replace it with original symbol.

* To construct 'follow' of a non terminal,
we find all productions for which this
symbol appears on the right side.

and then if it appears
any other production, we

$\text{follow}(A) \Rightarrow \text{follow}\{aAB\}$.

if the next symbol of the non terminal is :-

↓
first of B.

Nothing (or) Σ - include the answer of ~~original~~ follow of the

next symbol
after a.

Now, $\text{follow}(A) \Rightarrow \text{follow}\{a \in A\}$

nothing

so answer of follow(S) $\Rightarrow \{ \$ \}$ $\because S \rightarrow a$

↓
nothing.

$S \rightarrow aAB$ so, $\text{follow}(S) \Rightarrow \{ \$ \}$

follow(A) $\Rightarrow \{c, \$ \}$

$$f(B) = \{b\}$$

Ex: $S \rightarrow AB$
 ② $A \rightarrow a/\epsilon$
 $B \rightarrow b/\epsilon$

$\text{follow}(S) \rightarrow \{\$ \}$

$\text{follow}(A) \rightarrow \text{follow}(AB) \rightarrow \text{first}(B)$
 $\downarrow$
 NT

$\text{first}(B) \rightarrow \{b\} \Rightarrow \text{include}$
 $\{ \epsilon \} \Rightarrow \text{replace}$

$S \rightarrow A\epsilon \rightarrow A \downarrow$
 $\text{nothing} \Rightarrow \text{follow}(S) \rightarrow \{\$ \}$

$\therefore \text{follow}(A) \rightarrow \{b, \$ \}$

$\text{follow}(B) \rightarrow \text{follow}(AB \downarrow)$
 $\text{nothing} \Rightarrow \text{follow}(S) \rightarrow \{\$ \}$

$\therefore \text{follow}(B) \rightarrow \{\$ \}$

Q) $E \rightarrow TE'$
 $E' \rightarrow +TE' / \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' / \epsilon$
 $F \rightarrow id / (E)$

$NT: - E, T, E', T', F$
 $T: - +, *, id, (,)$

(Construct first, follow for all NTs)

$\text{first}(E) = \text{first}(TE')$
 $= \{id, (\}$

$\text{first}(T) = \text{first}(FT')$
 $= \{id, (\}$

$\text{first}(F) = \text{first}(id) = \{id\}$
 $= \text{first}((E)) = \{ (\}$
 $= \{ (, id \}$

$\text{first}(E') = \text{first}(+TE') = \{ + \}$
 $= \text{first}(\epsilon) = \{ \epsilon \}$
 $= \{ +, \epsilon \}$

$\text{first}(T') = \text{first}(*FT) = \{ * \}$
 $= \text{first}(\epsilon) = \{ \epsilon \}$
 $= \{ *, \epsilon \}$

$$\text{follow}(E) = \{ \$ \} \quad \because F \rightarrow (E)$$

$$= \text{follow}((E)) = \{ \rangle \}$$

$$= \{ \$, \rangle \}$$

$$\text{follow}(T) = \text{follow}(TE')$$

$$\because E \rightarrow TE'$$

$$\text{follow} = \text{first}(E') = \{ +, \epsilon \}$$

$$E \rightarrow T \quad \text{follow}(T) = \text{follow}(E) = \{ \$, \rangle \}$$

$$= \{ +, \$, \rangle \}$$

$$= \text{follow}(+TE') = \{ +, \$, \rangle \}$$

$$\therefore \text{follow}(T) = \{ +, \$, \rangle \}$$

$$\text{follow}(E') = \text{follow}(TE') \quad \because E \rightarrow TE'$$

$$\Rightarrow \text{follow}(E) = \{ \rangle, \$ \}$$

$$= \text{follow}(+TE')$$

$$\text{follow}(E') = \{ \rangle, \$ \}$$

$$\text{follow}(T') \Rightarrow T \rightarrow FT' \quad \text{follow}(T) = \{ +, \$, \rangle \}$$

$$= \text{follow}(T) = \{ +, \$, \rangle \}$$

$$\therefore \text{follow}(T') = \{ +, \$, \rangle \}$$

$$\text{follow}(F) \Rightarrow T \rightarrow FT'$$

$$\text{first}(T') = \{ * \}$$

$$T \rightarrow F \quad \text{follow}(T) = \{ +, \$, \rangle \}$$

$$\therefore \{ +, *, \rangle, \$ \}$$

$$\Rightarrow T' \rightarrow *FT'$$

$$\text{first}(T') = \{ * \}$$

$$\therefore \{ +, *, \rangle, \$ \}$$

$$T \rightarrow *F \quad \Rightarrow \{ +, \rangle, \$ \}$$

$$\therefore \text{follow}(F) = \{ +, *, \rangle, \$ \}$$

	First	Follow
E	{id, (}	{), \$ }
E'	{+, ε }	{), \$ }
T	{id, (}	{+,), \$ }
T'	{*, ε }	{+,), \$ }
F	{id, (}	{*, +,), \$ }

LL parsing table :- (Top down approach)

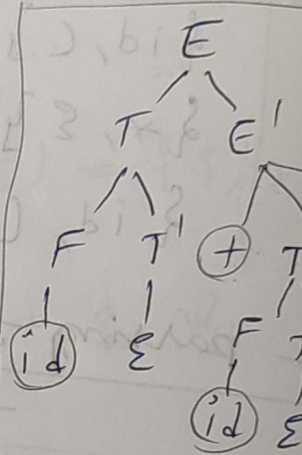
		TERMINALS					
		id	(	)	*	+	\$
Non T E R M I N A L S	E	$E \rightarrow TE'$ _x	$E \rightarrow TE'$ _x				
	E'			$E' \rightarrow \epsilon$ _y		$E' \rightarrow TE'$ _x	$E' \rightarrow \epsilon$ _y
	T	$T \rightarrow FT'$ _x	$T \rightarrow FT'$ _x				
	T'			$T' \rightarrow \epsilon$ _y	$T' \rightarrow *FT'$ _x	$T' \rightarrow \epsilon$ _y	$T' \rightarrow \epsilon$ _y
	F	$F \rightarrow id$ _x	$F \rightarrow (E)$ _x				

- we fill this table based on terminals. i.e each entry is filled with first production which on expansion gives the terminal respectively.
- we fill it for the symbols of the firsts of the non terminals. ✗
- If there is ε in firsts, then we fill it based on symbols of follow of that non terminal. ✓

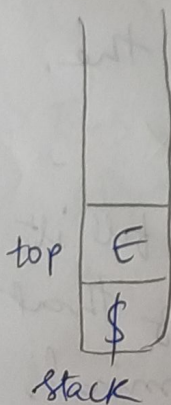
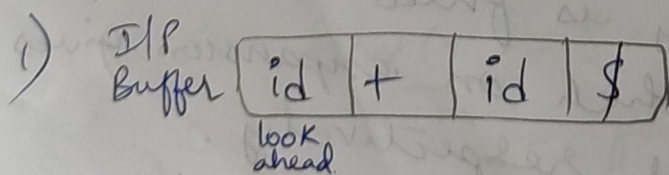
we check if we can generate it with given grammar using parsing

How it works:-

- we load the input string in the input buffer and add \$ symbol at the end of input buffer (queue) and also at the bottom of stack. (to know end state)
- Also insert start symbol of grammar into the stack.
- Now the parsing algorithm starts working.
- At the end if we reach end state i.e. $\$ == \$$, then it means that the string is accepted by grammar and it is syntactically correct.



Steps:-



Parsing Algorithm

LL parsing table

2) Find in table, the entry of E, id , it is $E \rightarrow TE'$, so insert TE' in reverse order in stack after popping E .

$\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

$\rightarrow \begin{array}{|c|} \hline T \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

Next, check entry of T, id in table, it is $T \rightarrow FT'$.

so pop & insert in reverse

- Also, each time, we check the top of stack and look ahead of buffer, whether they are same or not.

3) $\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

$\rightarrow \begin{array}{|c|} \hline F. \\ \hline T' \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

4) $\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

$\rightarrow \begin{array}{|c|} \hline id \\ \hline T' \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

here top of stack & look ahead of buffer is same so we pop top symbol and top-- and lookahead++

5) $\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

$\rightarrow \begin{array}{|c|} \hline \epsilon \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

check T' on $+$ in table and insert

$\therefore$ it is ϵ , we do nothing

7) $\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

6) $\overline{id \mid + \mid id \mid \$}$
 $\uparrow$

$\rightarrow \begin{array}{|c|} \hline + \\ \hline T \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

same, so: top-- lookahead++

$\rightarrow \begin{array}{|c|} \hline F. \\ \hline T' \\ \hline E' \\ \hline \$ \\ \hline \end{array} \Rightarrow \begin{array}{|c|} \hline id \\ \hline T' \\ \hline E' \\ \hline \$ \\ \hline \end{array}$

same again

8) $\text{id} \mid + \mid \text{id} \mid \$$

$\begin{matrix} \text{\textcancel{E}} \\ \text{E} \\ \$ \end{matrix}$ $\begin{matrix} T' \text{ on } \$ = \epsilon \\ E' \text{ on } \$ = \epsilon \end{matrix}$

9) $\text{id} \mid + \mid \text{id} \mid \$$

$\begin{matrix} \$ \end{matrix}$ same.

So we stop here.

(10) Therefore, given string is accepted by grammar and it is syntactically correct.

Stack	Input	Action
\$ E	<u>id</u> + id \$	$E \rightarrow TE'$
\$ E' T	<u>id</u> + id \$	$E' \rightarrow id$ $T \rightarrow FT'$
\$ E' T' F	<u>id</u> + id \$	$F \rightarrow id$
\$ E' T' id	<u>id</u> + id \$	top --, lookahead ++
\$ E' T'	<u>+</u> id \$	$T' \rightarrow \epsilon$
\$ E'	<u>+</u> id \$	$E' \rightarrow TE'$
\$ E' T +	<u>id</u> \$	top --, lookahead ++
\$ E' T	<u>id</u> \$	$T \rightarrow FT'$
\$ E' T' F	id \$	$F \rightarrow id$
\$ E' T' id	\$	top --, lookahead ++
\$ E' T'	\$	$T' \rightarrow \epsilon$
\$ E'	\$	$E' \rightarrow \epsilon$
\$	\$	

$\therefore$ Hence proved.

Precedence :-

Priority is from bottom to up in this grammar

Ex: - $2 + 3 * 4 = 14$

Here, grammar is :-

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow id / (E)$$

Precedence is. -

()

*

+

(Bodmas)

Ex: - $2 + 3 * 4 = 5 * 4 = 20$

Here grammar is :-

$$E \rightarrow TE'$$

$$E' \rightarrow *TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow +FT' / \epsilon$$

$$F \rightarrow id / (E)$$

Precedence is

()

+

*

NOTE: All grammars may not be accepted by LL parser. To find whether they can be accepted by LL parser or not, we construct first & follow then check the parsing table.

Ex: - $S \rightarrow aSbS$
 $\quad \quad bSaS$
 $\quad \quad \epsilon$

$$\text{first}(S) = a, b, \epsilon$$

$$\text{follow}(S) = b, a, \$$$

passing table:-

	a	b	\$
S	$S \rightarrow asbs$ $S \rightarrow \epsilon$	$S \rightarrow bsas$ $S \rightarrow \epsilon$	$S \rightarrow \epsilon$

Passing table

should clarify which production to use, to derive a terminal.

- But here, there are multiple entries so no clarity, therefore this grammar is not accepted by LL parser.

LR parser:- Bottom up parser.

4 types:- i) LR(0)

ii) SLR(1)

They use canonical collection of '0' items

iii) LALR(1)

iv) CLR(1)

They use canonical collection of '1' items

- All these parsers are also called as "shift reduce parsers".

Ex:- $S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

In this we use item construction of closure.

i.e we put '.' before each production which

is called item and based on the next symbol of '.', we create some productions/rules which is called construction of closure.

- Always first we give numbering to our grammar and create a new start state which leads to the original start state

$$1) S \rightarrow AA$$

$$S' \rightarrow S$$

$$2) A \rightarrow aA$$

$$3) A \rightarrow b$$

$\downarrow$ new start symbol
 $\downarrow$ original start symbol

Working:-

Step 1:- Introduce item:- $S' \rightarrow \cdot S$

Step 2:- Create/construct closure:-

check the symbol after $\cdot$ is T or NT
 if NT:- add its original productions.

(if T:- do nothing) $S' \rightarrow \cdot S$
 $S \rightarrow \cdot AA$

now repeat step 1 and step 2.

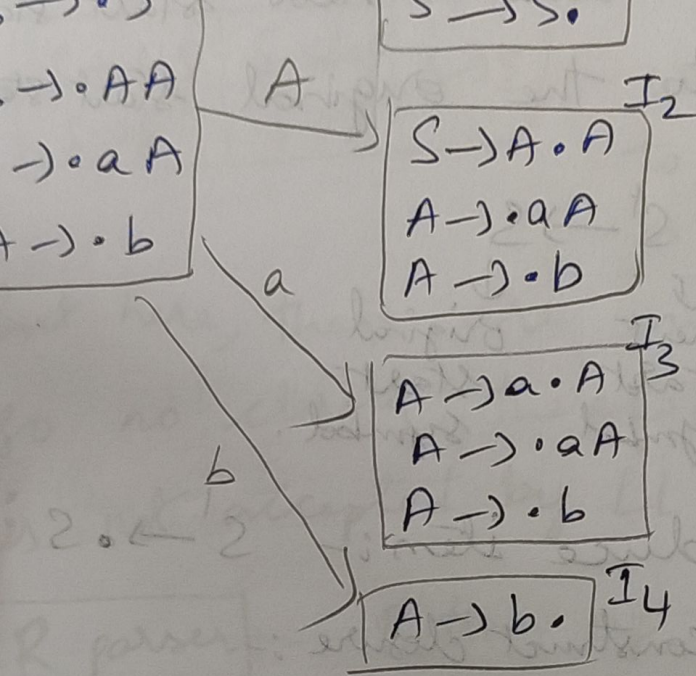
$\Rightarrow S' \rightarrow \cdot S$
 $S \rightarrow \cdot AA$
 $\downarrow$ NT
 $\Rightarrow$

$S' \rightarrow \cdot S$
$S \rightarrow \cdot AA$
$A \rightarrow \cdot aA$
$A \rightarrow \cdot b$

 $\Rightarrow$ This is called state 0 i.e. I_0

* We use all the productions of I_0 to create I_1 (state 1) by forwarding the ' $\cdot$ ' one step forward.

- Every time we get a production, we follow step 1, 2 i.e. item & closure
- For each production, different state is formed



i.e ϵ i.e T
 so do nothing.

here after $\cdot$,
 NT is there, so
 add its original prod.

here. also NT
 so add original.

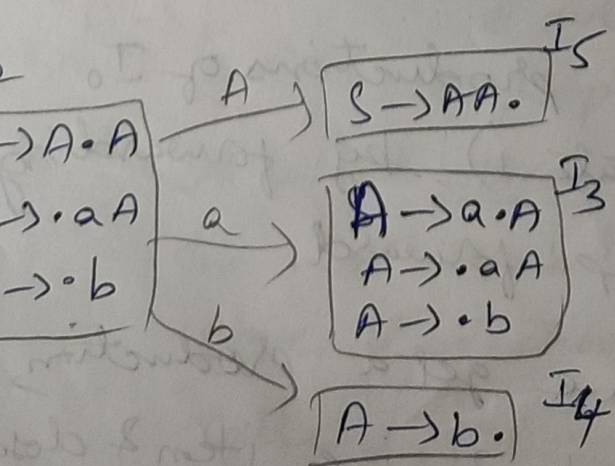
here nothing after.
 so do nothing

continue to do this process, until
 new states are generating.

The states which have $\cdot$ at the
 end position i.e right most position,
 they are called the End states.

we don't need to generate new
 states from end states.

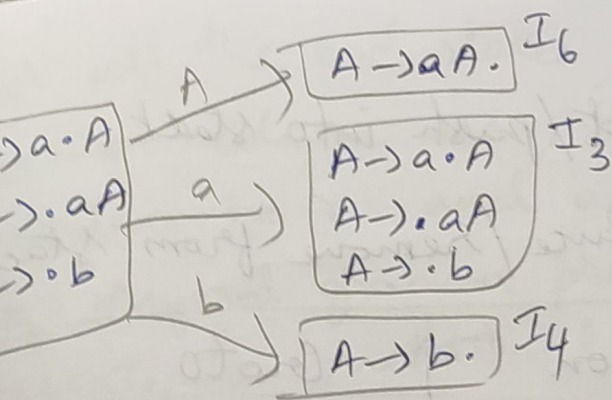
here, I_1 and I_4 are end states.



nothing.

this state is already
 available i.e I_3

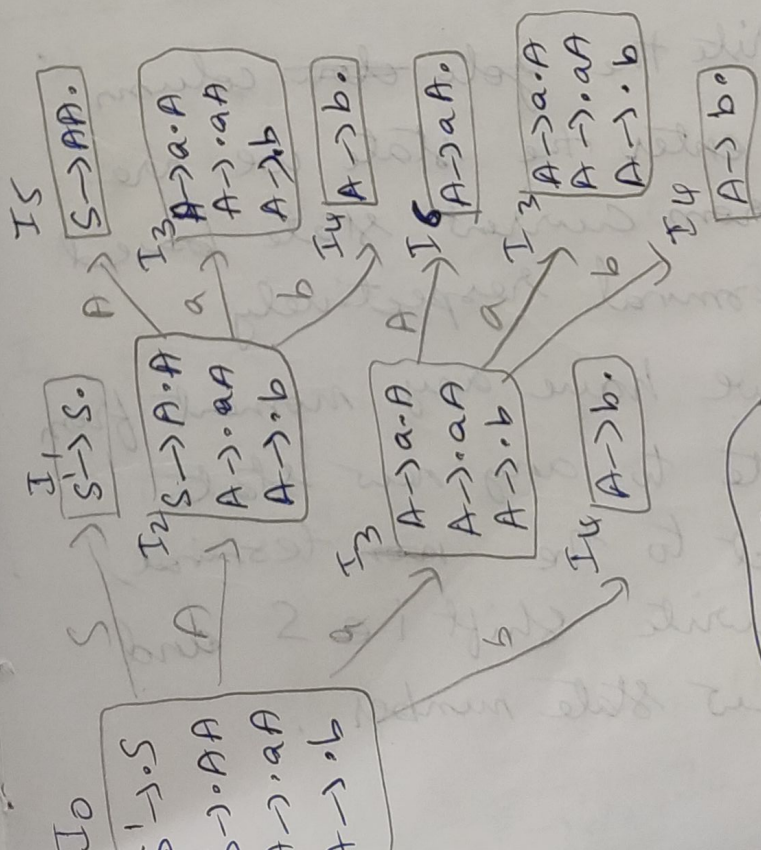
already present
 i.e I_4



As I_5 and I_6 are end states.
 So all states are processed now.

After writing whole explanation while creating new states, the below parse tree should be constructed in exam. So,

- Step 1:- construct states (canonical collections) theory of
- Step 2:- construct parse tree.



	first	follow
S	a, b	\$
A	a, b	a, b, \$

Parse table

Actions $\begin{cases} \text{shift/push into stack} \\ \text{reduce/remove from stack} \end{cases}$

States	Action			Goto	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S3	S4			5
3	S3	S4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

first we write the goto ~~column~~ column, where we enter the state we are going to from current state based on non terminal respectively.

Next, if we have any movement from current state to any new state with respect to the non terminal, then we write shift i.e S and also the new state number.

- For all the end states, the fields of action column are marked as reduce i.e r . and also the numbering of original grammar production of that state is mentioned.

(except for I_1 , because it's a new production, so it doesn't have numbering)

- so the block of I_1 row is called as Accept block.

- All the remaining unmarked blank/empty states are Error states, so, that particular string cannot be accepted.

9) 1) $E \rightarrow TE'$

2) $E' \rightarrow +TE'$

3) $E' \rightarrow \epsilon$

4) $T \rightarrow FT'$

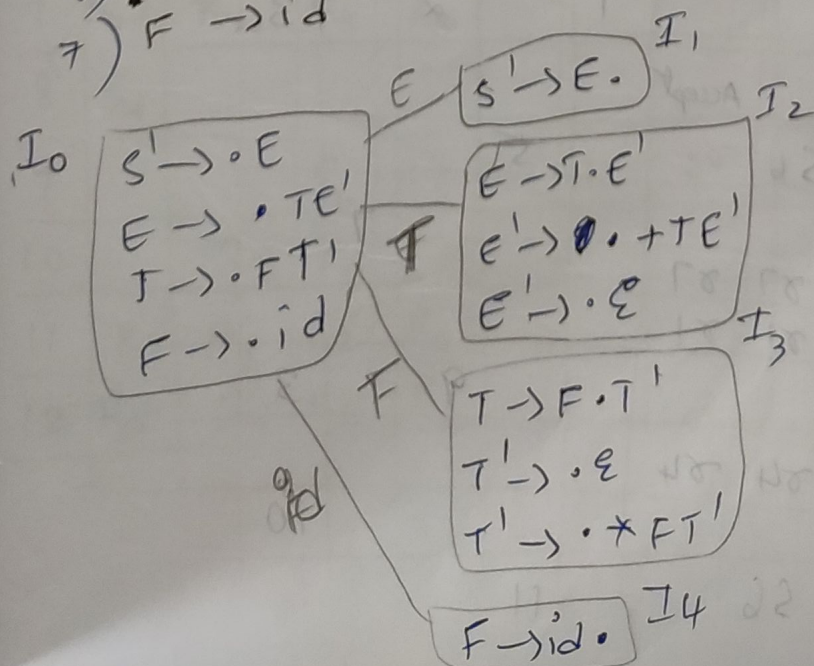
5) $T' \rightarrow *FT'$

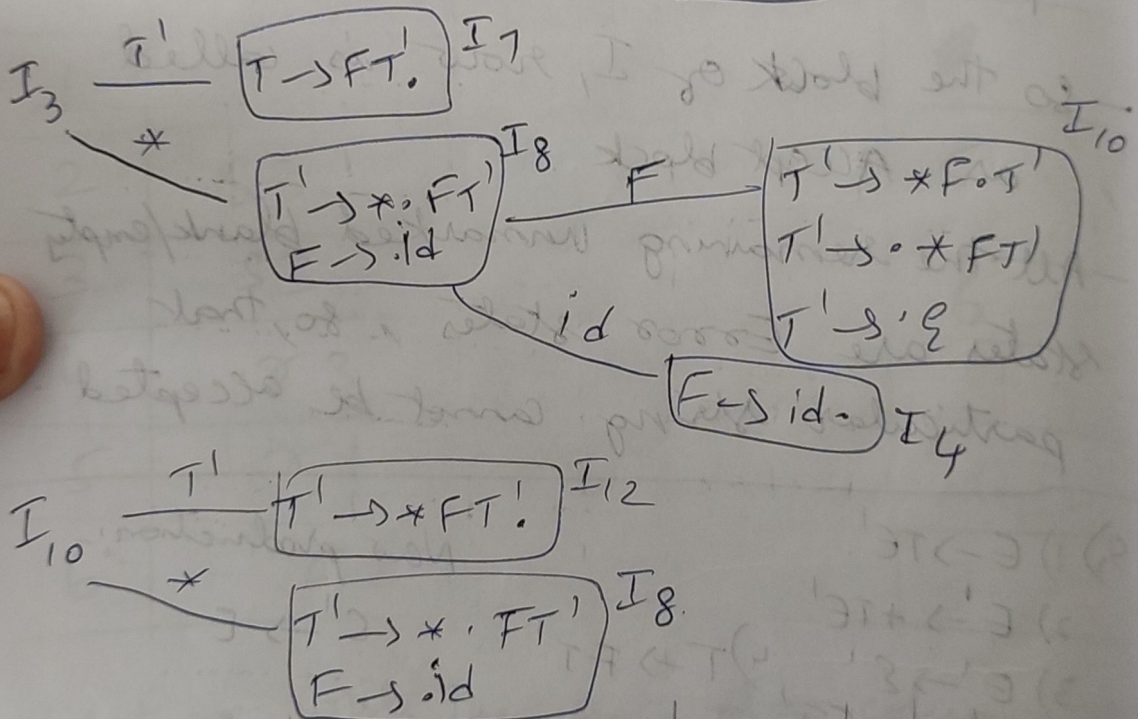
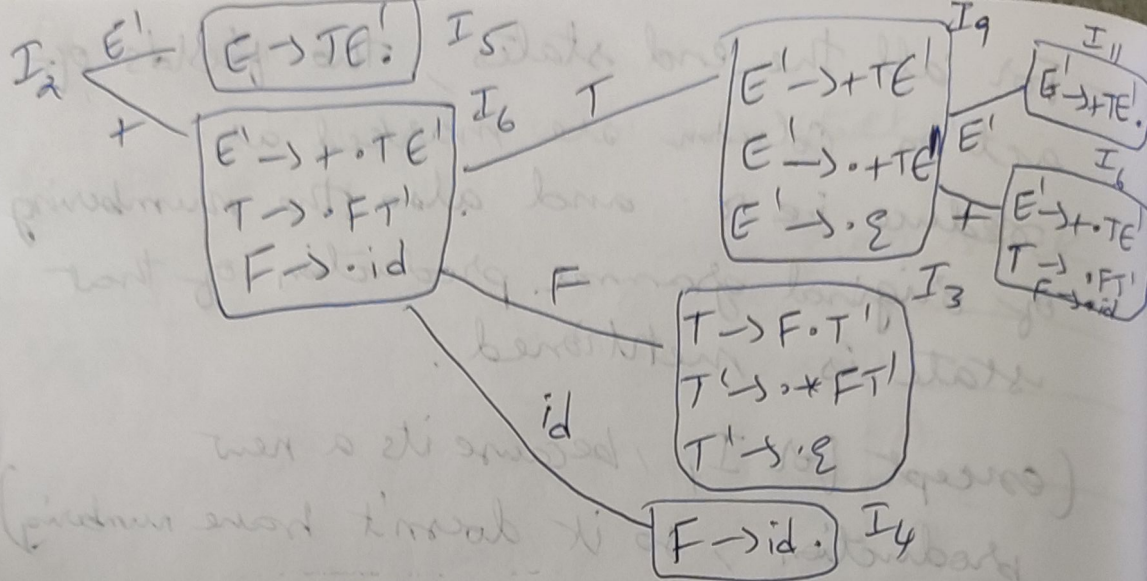
6) $T' \rightarrow \epsilon$

7) $F \rightarrow id$

New production:-

$S' \rightarrow E$





	id	*	+	\$	E	E'	T	T'	F
0	S ₄				1		2		3
1				Accept					
2	r ₃	r ₃	S ₆	r ₃		5			
3		S ₈						7	
4	r ₁	r ₇	r ₇	r ₇					
5	r ₁	r ₁	r ₁	r ₁					
6	S ₄						9		3
7	r ₄	r ₄	r ₄	r ₄					
8	S ₄								10
9			S ₆			11			

10		S8				12		
11	82	82	82	82				
12	85	85	85	85				

This grammar doesn't support LR grammar because there is moment from end states i.e. in same we should not get S and x

⇒ hence we use, **SLR parser** :-

	id	*	+	\$	E	E'	T	T'	F
0	S ₄				1		2		3
1			Accept						
2	R ₃	R ₃	S ₆ R ₃	R ₃		5			
3		S ₈	R ₆	R ₆			7		
4	R ₇	R ₇	R ₇	R ₇					
5	R ₁	R ₁	R ₁	R ₁					
6	S ₄						9		3
7	R ₄ R ₇	R ₄	R ₄	R ₄					
8	S ₄								10
9			S ₆	R ₃	11				
10		S ₈	R ₆	R ₆			12		
11	R ₂	R ₂	R ₂	R ₂					
12	R ₅	R ₅	R ₅	R ₅					

⊙ → empty state is Error Detection state

- Can't do shift & reduce at same time
- In such cases, it is called SR (shift reduce) error/conflict.
- Such grammars cannot be supported by LR_0 parsers.
- RR problem ($R_4 R_7$ or similar in single unit is table) also not supported by LR_0 parser.

$\Rightarrow$ In the above table, there are conflicts (RR & SR). To reduce this, we use $SLR(1)$ parser

- 1) Identify states that caused reduction (in the states the production that ended with $\cdot$)
- 2) In that state, identify the production that caused reduction.
- 3) For that production, identify the production symbol for eg:- $E \rightarrow \cdot$.
(E is production symbol)
- 4) Do follow (production symbol)
- 5) Mark the terminals that we get in follow (production symbol) as reduction states in the $SLR(1)$ parsing table.

6) The shift states and numbers are normally marked in the SLR(1) parsing table

The SLR(1) table is marked in pencil

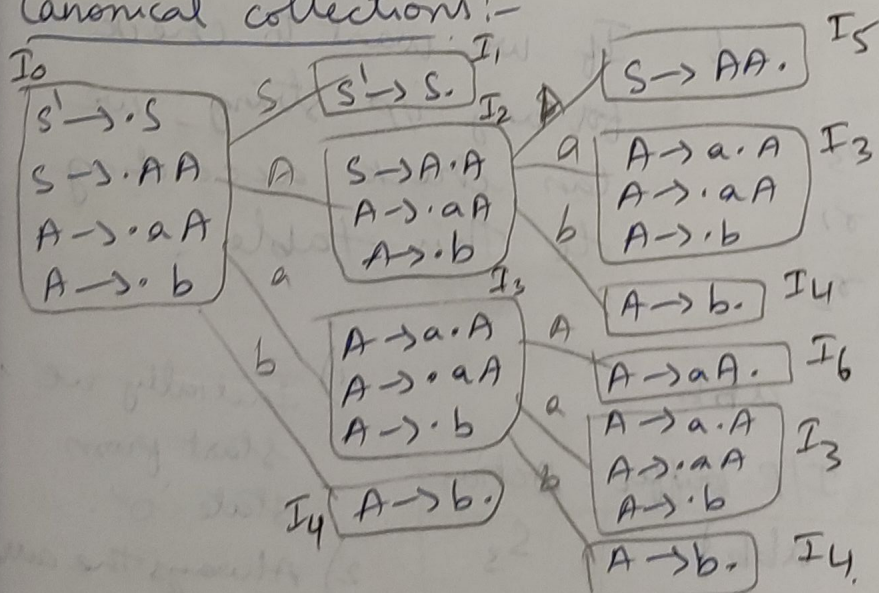
⇒ The error detection states increased in SLR table when compared to LR table.

⇒ hence, the present SLR table is more powerful as it can detect more no. of errors.

Q) $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

LR(0), SLR(1) ⇒ '0' lookahead
 (doesn't care about lookahead while reducing)
 LALR(1), CLR(1)
 ↓
 -1' look ahead.
 (dependent on 1 variable of look ahead)

Canonical collections:-



Now we draw LR(0) parsing table, as usual by marking S and r in actions and goto.

LR - 0

	a	b	\$	S	A
0	S ₃	S ₄		1	2
1			Accept		
2	S ₃	S ₄			5
3	S ₃	S ₄			6
4	r ₃	r ₃	r ₃		
5	r ₁	r ₁	r ₁		
6	r ₂	r ₂	r ₂		

SLR(1)

	a	b	\$	S	A
0	S ₃	S ₄		1	2
1			Accept		
2	S ₃	S ₄			5
3	S ₃	S ₄			6
4	r ₃	r ₃	r ₃		
5			r ₁		
6	r ₂	r ₂	r ₂		

Ex:- I/P = abb

stack	I/P buffer	Action
0	abb\$	S ₃
0a3	bb\$	S ₄
0a3b4	b\$	r ₃

If S_5 is end state, and its original production is) so we write r_1 in all blocks ~~by~~ but for SLR(1), for reduction, we first calculate follow() of $S \rightarrow AA$ production i.e follow(S) $\Rightarrow$ \$, so only where \$ is there we mark r_1 in 5th state. And for 4th, 6th state for $A \rightarrow aA$, $A \rightarrow b$, follow(A) is a, b, \$, so we mark for all

$\Rightarrow$ This is the SLR(1) parsing table.

If we want to check for any i/p string, we can check according to this table.

1) Initially we start from state '0'.

2) Always the current state should be on top of stack.

3) If action is S, then we push that symbol & state into stack.

0A3A6	b \$	σ_2
0A2	b \$	σ_4
0A2b4	\$	σ_3
0A2A5	\$	σ_1
05 1	\$	Accept

and move forward.

4) If action is σ , then we pop from stack state number length $\times 2$ times.

Ex:- $\sigma_3 \Rightarrow 3^{\text{rd}}$ prod

$A \rightarrow \underline{b} \Rightarrow \text{len} = 1$

$1 \times 2 = 2$ times pop

5) Continue whole process untill you get 1 on \$ i.e accept state (or) any empty block i.e error state.

And add LHS of that prod onto stack i.e A and top would be the state which we get on A from the state before it i.e 3.

A on 3 $\Rightarrow 6$.

$\Rightarrow$ Here string is accepted.

CLR(1) parsing :-

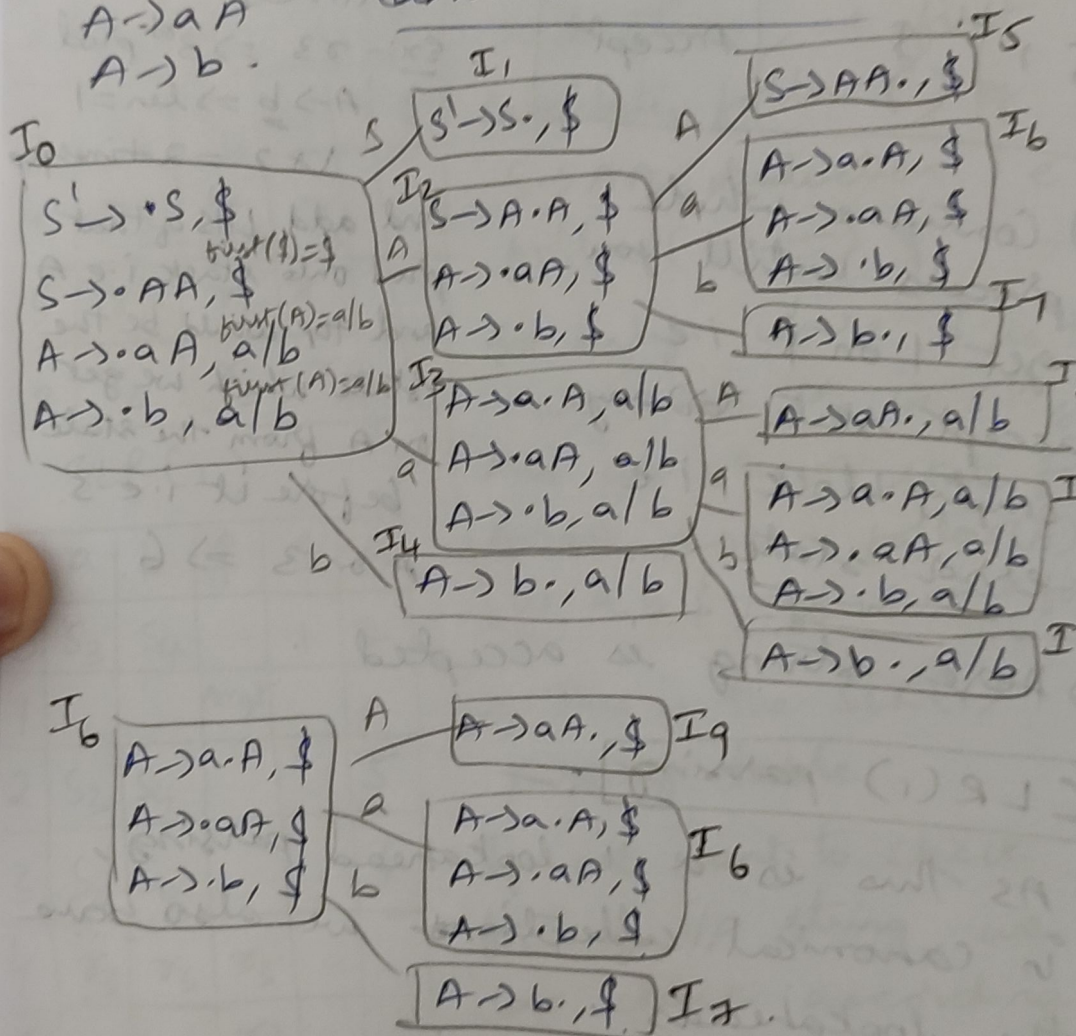
- As this is a '1' lookahead parsing, in canonical collections we also have the lookaheads.
- For starting symbol, \$ is lookahead.
- For next production, we consider look ahead as first of the string after Non terminal from which production is created.
- Even if production is same, if look ahead is diff, it is considered as a new state.

NOTE:-

No. of states:- $CLR \geq SLR(1), LR(0)$
 $LALR(1) = SLR(1), LR(0)$

Q) $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

Canonical collections:-



So now in CRL table, for the new state we write:-

	a	b	\$
4	r_3	r_3	
5			r_1
7			r_3
8	r_2	r_2	
9			r_2

* CLR is the most powerful parser.

CLR supports all grammars which are supported by LR, SLR and LALR. but vice versa not true.

* All the parsers only work on context free grammar (CFG) but every CFG may not be parsed by any of the ~~grammars~~ parsers.
(100% parsing is not possible for every CFG)

* Ambiguous grammar is not parsable but operator precedence parser can parse few ambiguous grammars. $\Rightarrow$ (bottom up)

LALR(1) parser:-

- First it is mandatory to construct CLR(1) in LALR(1) $\Rightarrow$

$\Rightarrow$ previous example:-

states whose productions are same except look ahead:-

$I_3, I_6 \Rightarrow I_3$ and $I_6 \Rightarrow I_{36}, I_{36}$.

$\Rightarrow$ $\boxed{\begin{array}{l} A \rightarrow aA, a/b/\$ \\ A \rightarrow \cdot aA, a/b/\$ \\ A \rightarrow \cdot b, a/b/\$ \end{array}} I_{36}$

I_4 and $I_7 \Rightarrow I_{47}, I_{47} \Rightarrow \boxed{A \rightarrow b \cdot, a/b/\$} I_{47}$

I_8 and $I_9 \Rightarrow I_{89}, I_{89} \Rightarrow \boxed{A \rightarrow aA, \$} I_{89}$

* If there are duplicate rows, combine them. $\boxed{r_1}$ and $\boxed{r_2} \Rightarrow \boxed{r_3}$.

* But if conflict comes i.e

$\boxed{r_3}$ and $\boxed{r_4} \Rightarrow$ invalid.

i.e can't parse.

	a	b	\$	S	A
0	S ₃₆	S ₄₇		1	2
1			Accept		
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	τ_3	τ_3			
5			τ_1		
36	S ₃₆	S ₄₇			89
47			τ_3		
89	τ_2	τ_2			
89			τ_3		

duplicate,
delete any one

combine.

combine.

2, S₃₆ remove one.

2, S₄₇ [47 | τ_3 | τ_3 | τ_3 | | A]

S₈₉ [89 | τ_2 | τ_2 | τ_2 | |]

ENV [36 | S₃]

[36 | S₇]

$\Rightarrow$ [36 | S₃ / S₇]

we can say
that the
grammar is
not parsable.

UNIT-3

* After syntax analyzer, to check the meaningfulness of grammar, we use semantic analyzers.

- These semantic analyzers have grammar + action. They are called Syntax directed translation/definition (SDT/SDD)

- They check meaningfulness, type checking, evaluation etc, precedence checking.

- Generally, we use these in YACC programs.

Ex- $E \rightarrow E + T$ $\left\{ \begin{array}{l} \text{action} \\ \text{part} \end{array} \right\}$
 $\left(\begin{array}{l} \text{val} \\ \text{type} \end{array} \right) \left(\begin{array}{l} \text{val} \\ \text{type} \end{array} \right) \left(\begin{array}{l} \text{val} \\ \text{type} \end{array} \right)$ (To reduce $E+T$ to E).

- Based on our requirement, we can write one or multiple SDTs for a grammar.

- To write multiple SDTs, we have to add an attribute to the grammar for type checking (int/val/float etc).

$\Rightarrow$ That's why this is called Attributed grammar as there is an attribute attached to every variable in the grammar.

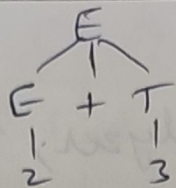
$\Rightarrow$ Grammar + Attributes = SDT.

* Semantic analyzers create syntax trees

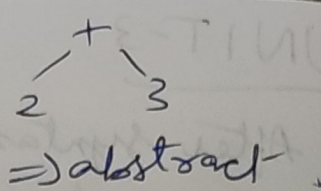
- A syntax tree with variables and final values is called Concrete syntax Tree

- A syntax tree with only final values is called abstract syntax tree

Ex: $E \rightarrow E + T$
 $2 + 3$



$\Rightarrow$ concrete



$\Rightarrow$ abstract

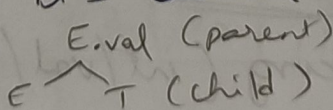
* SDTs can do following operations; —

- Type checking
- Evaluation
- Syntax Tree
- Posting notation
- Prefix notation
- 3 address code.

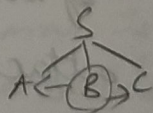
— Based on the action part, SDTs perform any of the above operations apart from checking meaning of grammar.

* YACC programs internally use lex concepts so they can also perform token/note/line splitting and precedence checking etc.

* If a parent node gets its value from child node then it's called Synthesis attributes



* If a node gets its value from its sibling nodes (left/right) then those attributes are called Inherited attributes



* Based on these attributes and their functions, the attributed grammar is divided into i) S-attributed grammar ii) L-attributed grammar.

attributed grammar

by synthesized attributes ①

The RHS of the grammar must be completely parsed that it has final values which can be given to parent i.e. LHS. So action part here is written at the end. ②

Here, Bottom up procedure is used to parse the syntax tree. ③

$\} \Rightarrow$ action part

L-attributed grammar

Both synthesized and inherited attributes but node can take value from its left sibling only. (only taken from parent, left sibling, not dependent on right sibling) ①

here the action part can be written at any position.

if we want value of B :- $A \rightarrow \{ \} BC$

if we want value of C :- $A \rightarrow B \{ \} C$. ②

Here, Depth first approach is followed from left to right ③